Let the software log is S_L, event log is E_L, method call m, and event is e, $\#_n(e)$ is the value of attribute n for event e, and $\#_n(m)$ is the value of attribute n for method m. COM be the component of the software system. SD be the software execution data.

Table 2: Compliance specifications

✓- directly supported ∼ - Derivable × - not supported			
ID	Description	Study	Status
1	The software event log is a finite multi-set of traces.	[13], [20], [7], [10],[30]	✓
2	A trace is a finite sequence of unique events.	[13], [14], [16], [30], [33], [32]	∼
3	Each event can specify number of attributes.Mandatory attributes are an event name and timestamp. Events are ordered by logging time in each trace.	[30]	✓
4	Any event in the method call universe must include: method call ID, instance, class, caller method, caller class, caller object, start time, end time, transaction type, and owning component.	[13], [16], [17], [15]	✓
5	For any method call, nested method calls should be able to capture.	[32]	✓
6	Each event should have a unique classifier, which is method call ID and transaction type.	[13]	✓
7	Event log is consistent if and only if there are two events $e\&e'$ such that $\#_{method}(e) = \#_{method}(e'), \#_{instance}(e) = \#_{instance}(e'), and\ \#_{transactionType}(e) \neq \#_{transactionType}(e')$.	[13]	∼
8	For any two methods m_i and m_j, if $\#_{call}(m_i) = m_j$, then m_i is the caller. If $\#_{call}(m) = \emptyset$, then m is *main* method.	[14], [16], [17], [19], [26]	✓
9	Every method call is associated with a method with an object instance, and each instance originates from a class.	[14], [19]	✓
10	Each method call is mapped to a start and end timestamp.	[14], [15], [19]	✓
11	Event class names are unique identifiers. If two event classes $C1\&C2$ have the same names $C1.name = C2.name$, they represent the same set of events $C1.events = C2.events$. If $C1$ is a super class of $C2$, then $C2.events \subseteq C1.events$.	[30]	∼
12	Each method call is linked to a component instance, whose execution data includes all calls on objects of its classes.	[14], [16], [17], [26]	∼
13	Component instance set is all objects instantiated from its component. No shared objects and connections across objects belong to different component instances.	[14], [16]	✓
14	The union of all component instances is equal to the entire object set.	[14],[16]	∼
15	The object on which the method call m executes must be a member of the component instance to which m is assigned.	[14]	∼
16	Event ordering via directly followed, overlap, and contains relationships can be used to infer direct succession, causality, concurrency, choice, and nesting.	[29], [13]	∼
17	Frequency ratios for directly follows, overlaps, and containment relations between method calls can be computed to quantify their occurrence and dominance within the event log.	[13]	∼

18	Method interactions can be modeled as an object interaction graph, where nodes represent class object instances that execute at least one method, and edges represent method calls between these objects.	[14],[16], [17]	~
19	The top-level method set of a component can be derived as the set of methods that are called by methods belonging to other components.	[17]	~
20	The main event set comprises method calls triggered by external components. The main event log includes, for each trace, only the method calls belonging to this set.	[14],[16]	~
21	Nested event set contains two events *e*&*ne* belongs to the same trace, where the number of caller contexts of *e* equals the number of callee contexts of *ne*, which can be used to represent the internal structure of a component.	[14],[16]	~
22	The invoked event set consists of method calls directly invoked by the nested event set. These represent the child events of the corresponding nested method calls.	[14],[16]	~
23	A hierarchical software event log is constructed by starting with main events and recursively adding their callee events as child nodes, forming a tree-like structure.	[14],[16]	~
24	Candidate interface set of a component, identifies the methods accessed by external components or objects. A method *m* of component *C* is a top-level method if it is invoked by a method outside *C*. For each external caller *n*, the set of top-level methods invoked by *n* forms a candidate interface for *n*. The collection of all such sets, one per external caller, forms the candidate interface set of component *C*	[17], [26]	~
25	Method calls should associate with input parameter object sequences such that for each method call, there is a sequence of input parameter objects of the instance of the method	[15], [19]	✓
26	Given software execution data, it should be able to derive: the set of executed classes, executed method set of a class, input parameter set of every executed method, and invoked method set of each method.	[15], [19]	~
27	To enable process mining, events generated from a software system must include the following information: (i) the start and end timestamps of a method execution, (ii) the node where the event was generated, (iii) execution thread, (iv) the join point that triggered the event, (v) communication resource details (e.g., TCP/IP connection, IP address, and port), and (vi) the instrumented join point.	[12]	~
28	In a distributed setting, a node (web server) has multiple threads (node instances executed by threads). Each communication resource (channel) belongs to two different nodes. For any two different nodes, their set of communication resources and node instances must be disjoint.	[12]	✓
29	Given a communication resource and a communication interval, the set of events associated with the resource must be derivable.	[12]	✓
30	Two events $x, y \in E_L$ are directly related iff either: i) x and y generated by same thread. x executes within the duration of y. ii) Event y starts before event x, and the resources of x and y refer to the same communication channel.	[12]	~

31	Number of occurrences of a particular sequence in the event log –support, can be derived from the event log	[5], [22]	✓
32	Likelihood of another event occurring after a predefined sequence in the event log–confidence, can be derived from the event log.	[22]	✓
34	All events in a sequence are indexed by their position in the sequence.	[22]	✓
34	For any method, the interface set invoked by the method is defined as the interaction model of the method.	[26]	✓
35	The sequence of method calls within the execution of a method, m, and their receiver objects, form an object collaboration of the method.	[28]	✓
36	Objects share common method sets, which creates the concept of a role—defined as the set of methods invoked on an object during a collaboration. For an object o, its role is the set of method signatures s called on o within a call sequence S.	[28]	×
37	Given a hierarchical event log, the following properties should be derivable for each node: its depth (number of steps from root), parent node, event class, class of the event, set of leaf nodes, and the path from the root to the node.	[30]	~
38	Hierarchical Software Execution Event Log, $HLOG = (mainLog, Mapping : event- > subLog, layer)$. The main event log contains the top-level execution events. Mapping contains the association of nested events in the main log to the sub log. Layers indicate the depth or number of nesting levels in the event hierarchy.	[32]	✓
39	An interaction log can be generated from software log data using the set of objects and methods, capturing all possible finite sequences of method invocations.	[7]	✓
40	Hierarchical interaction model is a five-tuple with–set of objects, containers, object to container mapping function, set of messages, and allowed interactions.	[7]	✓
41	The events generated through instrumented concurrent program should include $read(T, R)\&write(T, R)$ which denotes the read and write operations performed by thread T on shared resource R.	[3]	×
42	The events generated through instrumented concurrent program should include $acquire(T, L)\&release(T, L)$ representing thread T acquiring and releasing lock L.	[3]	×

Table 1: Software log properties used for mining

Property	Description	Study	SEED
Timestamp Start/end time Duration	Timestamp is the time of invoking a method. Start and end timestamp defines the invoked time and completed time of methods and their difference used as duration.	[31], [29], [13], [14], [16], [19], [1], [7], [5], [11], [10]	✓
Session ID	Unique identifier of the session.	[31]	✓
Trace ID	Unique identifier of the request.	[29], [13], [6], [10], [8], [9]	✓
Method/ Callee method	Method being invoked - This can include the fully qualified name, method name, class, package, and input parameters and return type.	[31], [13], [14], [18], [16], [17], [19], [1], [21], [7], [2], [4], [6], [23], [24], [25], [26], [27], [32], [28]	✓
Method ID	Unique identifier assigned to method.	[13], [14], [17], [26]	✓
Method call instance/ Callee object/ target/ receiver	The unique hash code identifier for the runtime object of the class, which the triggered method belongs.	[13], [14], [18], [16], [17], [19], [7], [2], [26], [28]	✓
Class/ Callee class/ component/ Type of callee	The class of the invoking method.	[31], [13], [18], [16], [21], [23], [24], [25], [32], [28]	✓
Transaction type	Indicates the start, complete state of invoking method.	[13]	✓
Span ID/ Parent Span ID	Used by endpoint API calls to identify the caller and callee services.	[29]	✓
Caller class/ Type of caller	Class of the method initiates the call.	[23], [24], [25], [28]	✓
Caller object/ source/ sender	Unique hash code of the runtime class object, which initiates the call.	[14], [16], [17], [19], [7], [2], [26], [28]	✓
Caller method	Method initiates the call.	[14], [16], [17], [19], [26], [32]	✓
Input parameters	Input parameters of the method as a set of objects.	[19], [7], [28]	✓
Thread/Process identifier	Thread or process ID associated with the method being invoked.	[21], [7]	✓
Constructor context	Method calls that are made in the constructor.	[21]	✓
NestedCalls	The method being invoked contains other method invocations or not.	[32]	✓
Source code location	Source code position of the method - line number.	[28]	✓
Object collaboration	Sequence of method calls within a method.	[28]	×
Return parameters	Return parameters of the method being called.	[28]	✓
Created objects	Objects that are initiated inside the method being invoked.	[12]	×
Read/write access to objects	Read and write access to objects and fields inside the method.	[12], [27]	×
Affecting attributes	Updated attributes inside the method being invoked.	[12]	×
Temporal order of method calls	Ordered list of method calls sorted by timestamp.	[21]	✓
Method context as ancestors	Invoked methods within the method as a hierarchy.	[21]	✓
Thread read/write operations	Read/write access to resources.	[3]	×
Thread lock acquire/release	Acquired and released locks by the thread.	[3]	×
Depth	Distance from root to node in the hierarchical calling tree.	[30]	✓
Sequence of method calls	Invoked sequence of method calls as traces.	[4], [6], [33]	✓
User ID	User triggered the method call/ request.	[5]	✓

References

[1] Brünink, M., Rosenblum, D.S.: Mining performance specifications. In: FSE, pp. 39–49 (2016)

[2] Fahland, D., Lo, D., Maoz, S.: Mining branching-time scenarios. In: ASE, pp. 443–453 (2013)

[3] Ferrando, A., Delzanno, G.: Hypermonitor: A python prototype for hyper predictive runtime verification. In: Reachability Problems, pp. 171–182 (2023)

[4] Gabel, M., Su, Z.: Symbolic mining of temporal specifications. In: ICSE, pp. 51–60 (2008)

[5] Gadler, D., Mairegger, M., Janes, A., Russo, B.: Mining logs to model the use of a system. In: 2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), pp. 334–343 (2017)

[6] Jeevarathinam, R., Thanamani, A.S.: Transaction mapping based approach for mining software specifications. In: NaBIC, pp. 1596–1599 (2009)

[7] de Jong, T., van der Werf, J.M.E.M.: Process-mining based dynamic software architecture reconstruction. In: Proceedings of the 13th European Conference on Software Architecture - Volume 2, pp. 217–224 (2019)

[8] Kalsing, A., Iochpe, C., Thom, L., Nascimento, G.: Evolutionary learning of business process models from legacy systems using incremental process mining. ICEIS pp. 58–69 (2013)

[9] Kalsing, A.C., Iochpe, C., Thom, L.H., do Nascimento, G.S.: Re-learning of business process models from legacy system using incremental process mining. In: Enterprise Information Systems, pp. 314–330 (2014)

[10] Kalsing, A.C., Nascimento, G.S.d., Iochpe, C., Thom, L.H.: An incremental process mining approach to extract knowledge from legacy systems. In: 2010 14th IEEE International Enterprise Distributed Object Computing Conference, pp. 79–88 (2010)

[11] Kato, K., Kanai, T., Uehara, S.: Source code partitioning using process mining. In: Business Process Management, pp. 38–49 (2011)

[12] Leemans, M., van der Aalst, W.M.P.: Process mining in software systems: discovering real-life business transactions and process models from distributed systems. In: Proceedings of the 18th International Conference on Model Driven Engineering Languages and Systems, pp. 44–53 (2015)

[13] Liu, C.: Automatic discovery of behavioral models from software execution data. IEEE Transactions on Automation Science and Engineering (2018)

[14] Liu, C.: Discovery and quality evaluation of software component behavioral models. IEEE Transactions on Automation Science and Engineering **18**, 1538–1549 (2021)

[15] Liu, C., van Dongen, B., Assy, N., M. P. van der Aalst, W.: A framework to support behavioral design pattern detection from software execution data. In: Proceedings of the 13th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE 2018), pp. 65–76 (2018)

[16] Liu, C., van Dongen, B., Assy, N., van der Aalst, W.M.: Component behavior discovery from software execution data. In: 2016 IEEE Symposium Series on Computational Intelligence (SSCI), pp. 1–8 (2016)

[17] Liu, C., van Dongen, B., Assy, N., van der Aalst, W.M.P.: Component interface identification and behavioral model discovery from software execution data. In: Proceedings of the 26th Conference on Program Comprehension, pp. 97–107 (2018)

[18] Liu, C., van Dongen, B., Assy, N., van der Aalst, W.M.P.: A general framework to detect behavioral design patterns. In: ICSE, pp. 234–235 (2018)

[19] Liu, C., van Dongen, B.F., Assy, N., van der Aalst, W.M.P.: Detecting behavioral design patterns from software execution data. In: Evaluation of Novel Approaches to Software

Engineering, pp. 137–164 (2019)
[20] Liu, C., Wang, S., Gao, S., Zhang, F., Cheng, J.: User behavior discovery from low-level software execution log. IEEJ Transactions on Electrical and Electronic Engineering **13**, 1624–1632 (2018)
[21] Lo, D., Khoo, S.C.: Smartic: towards building an accurate, robust and scalable specification miner. In: Proceedings of the 14th ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 265–275, SIGSOFT '06/FSE-14 (2006)
[22] Lo, D., Khoo, S.C., Liu, C.: Mining temporal rules for software maintenance. Journal of Software Maintenance and Evolution: Research and Practice pp. 227–247 (2008)
[23] Lo, D., Maoz, S.: Specification mining of symbolic scenario-based models. In: PASTE, pp. 29–35 (2008)
[24] Lo, D., Maoz, S.: Mining hierarchical scenario-based specifications. In: 2009 IEEE/ACM International Conference on Automated Software Engineering, pp. 359–370 (2009)
[25] Lo, D., Maoz, S.: Scenario-based and value-based specification mining: better together. In: ASE, pp. 387–396 (2010)
[26] Lu, T., Liu, C., Duan, H., Zeng, Q.: Mining component-based software behavioral models using dynamic analysis. IEEE Access **8**, 68883–68894 (2020)
[27] Martinelli, F., Mercaldo, F., Nardone, V., Orlando, A., Santone, A., Vaglini, G.: Model checking based approach for compliance checking. Information Technology and Control pp. 278–298 (2019)
[28] Pradel, M., Gross, T.R.: Automatic generation of object usage specifications from large method traces. In: ASE, pp. 371–382 (2009)
[29] Raj, V., Chander, G.P.: Monitoring of microservices architecture based applications using process mining. In: 2022 9th International Conference on Computing for Sustainable Global Development (INDIACom), pp. 486–494 (2022)
[30] Stepanov, E.V., Mitsyuk, A.A.: Extracting high-level activities from low-level program execution logs. Automated Software Engineering **31** (2024)
[31] Taibi, D., Systä, K.: From monolithic systems to microservices: A decomposition framework based on process mining. In: Proceedings of the 9th International Conference on Cloud Computing and Services Science, pp. 153–164 (2019)
[32] Tang, Y., Li, T., Zhu, R., Du, F., Wang, J., Ma, Z., Ortin, F.: A discovery method for hierarchical software execution behavior models based on components. Sci. Program. **2021** (2021)
[33] Zhong, X., Zhang, N., Duan, Z.: An approach for automatically generating traces for python programs. In: 2022 9th International Conference on Dependable Systems and Their Applications (DSA), pp. 262–268 (2022)